University of Lancashire

Coursework Cover Sheet

Students should complete the input fields contained in this form and attach it in front of your formal assessment submission. All fields within this form are required. Please ensure that check boxes and radio buttons are appropriately selected. The last three questions are just for you to personally consider.

Department and assessment information:

School Name: University of Lancashire

Assessment title: Finite State Machines, Microcontroller and FPGA Comparison

Course Title: Electronic Engineering
Module Title: Digital Systems
Module Code: EL2242
Year of Study: 2026
Student Number: Mo Yan: [redacted]; Lowlife555: [redacted]

Academic Misconduct / Plagiarism Declaration

By attaching this front cover sheet to my assessment I confirm and declare that **I am the sole author of this work**, except where otherwise acknowledged by appropriate referencing and citation, and that I have taken all reasonable skill and care to ensure that no other person has been able, or allowed, to copy this work in either paper or electronic form, and that prior to submission I have read, understood and followed the University regulations as outlined in the Academic Integrity Policy and Procedure for Academic Misconduct

Have you checked the following? This will help your assessment achievement.

I have applied the learning outcomes for this module ☑
I have checked for Academic Integrity via Turn-it-in ☑
I have used references in accordance with instructions in the Assessment Brief ☑
I have proofread my work for spelling, grammar and punctuation. ☑
I have checked that the word count/size of this submission accords with the guidance provided in the Assessment Brief.☑
I have followed the guidance in the Assessment Brief and have not used AI or translation software to boost my grade unfairly. ☑

Well-being

We wish to support any student who is experiencing mitigating circumstances which prevents students from performing to the best of their ability when completing or submitting assignments. If you are experiencing such circumstances, then you may apply for Mitigating Circumstances. Wherever possible this must be done prior to handing in your assignment.
Do you need to apply for mitigating circumstances for this assignment No
Please refer to the Mitigating Circumstances Policy

Questions you may wish to consider:

1. Have I allowed sufficient time to prepare this assessment?
2. Have I reflected on previous feedback and made improvements in accordance with advice?
3. What grade am I expecting?

University of Lancashire

Finite State Machines, Microcontroller and FPGA Comparison

By

Mo Yan and Lowlife555

This paper is the major assignment for the EL2242 course

Electrical Engineering
University of Lancashire, UK

March, 2026
Comment

University of Lancashire

1. Theoretical Research on Finite State Machine (FSM).....4

1.1 Introduction.....4

1.2 Background of FSM.....4

1.3 Types of FSM: Moore and Mealy.....4

1.4 Advantages and disadvantages in system design.....5

1.5 Difference: FPGA vs Microcontroller.....5

2. Design Implementation and Development Instructions.....6

2.1 Overall System Overview.....6

2.2 Register-Level Development Implementation Based on STM32.....7

2.3 Hardware Logic Implementation Based on FPGA (Verilog HDL).....9

3. Comparison of development processes.....11

3.1 STM32 Microcontroller Development Process.....11

3.2 Altera FPGA Development Process.....12

4. Presentations of the Implemented Systems.....14

5. Reference List.....14

1. Theoretical Research on Finite State Machine (FSM)

1.1 Introduction

In embedded system design, we often need deal with tasks with specific sequential rules, for example the traffic light control system in this experiment. Finite State Machine (FSM) is a kind of logic model use to describe the system jump between limited number of states according to external input signals. According to Mano and Ciletti (2018), a sequential circuit can be defined as a finite state machine when it can be fully expressed by a state diagram with a finite number of states. For a traffic light, it cannot be in "green" and "red" state at the same time. The traffic light will according to input signals such as pedestrian crossing requests, real-time traffic flow and so on, cycle in the preset state sequence.

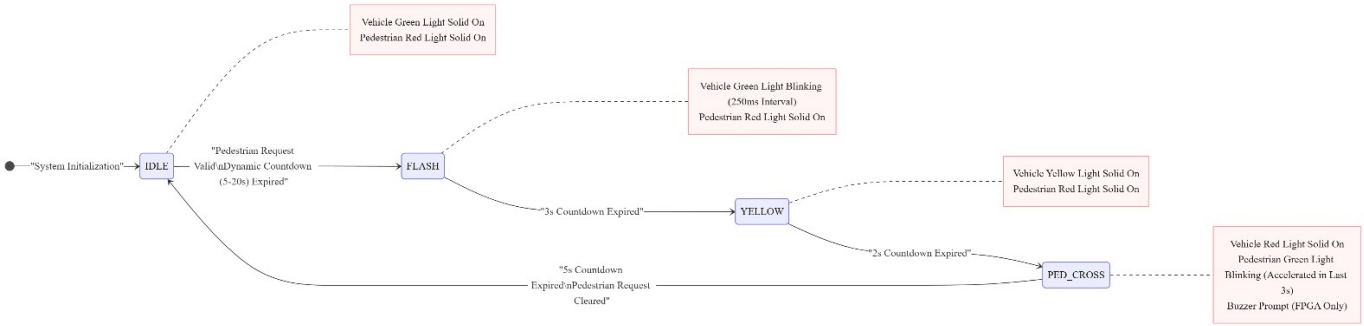


Figure 1 Traffic light system logic

1.2 Background of FSM

Base on the basic definition in the previous section, the core concept of FSM originates in early of computer science. Simply speaking, any FSM has existed five parts: States, Inputs, Outputs, Transitions and Initial States. In the digital logic design, FSM is the brain for system, and Roth and Kinney (2020) pointed out that state machines are at the core of the vast majority of digital systems, providing a structured approach for designing complex sequential logic. Through decompose complex behaviour into discrete states, engineers can decrease largely the risk of logic confuse, ensure system at all time execute according to the regulated behaviour. In this experiment, FPGA use Verilog to build these logics directly at hardware circuit level; while STM32 is through CPU execute command stream, realize control on the base of time-sharing multiplexing.

1.3 Types of FSM: Moore and Mealy

Base on the output signal's way in hardware design, FSM mainly divide into two types. Their core difference is: "In a Moore machine, the outputs are a function of the current state only, whereas in a Mealy machine, the outputs depend on both the current state and the inputs" (Pedroni, 2013). In this traffic light function realize, we adopt Moore type state machine. This is because Moore type state machine's output after pass clock edge is stable, will not produce influence because of input's tiny fluctuation (can avoid button shake or bottom noise generated by potentiometer), this is more stable and reliable for drive nixie tube and buzzer.

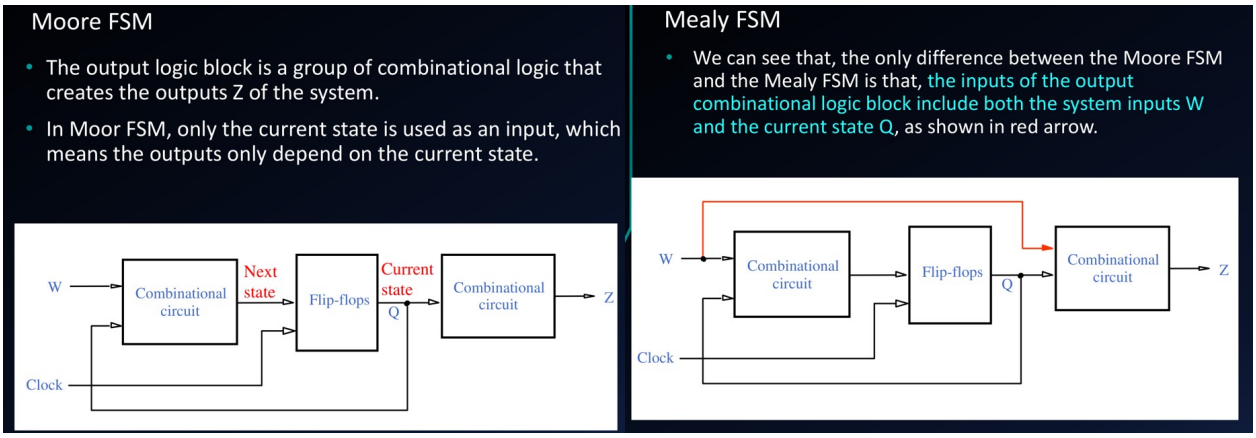


Figure 2 (Left side) Moore diagram, (Right side) Mealy diagram

1.4 Advantages and disadvantages in system design

Use FSM in embedded control system development has very big advantage, Wolf (2012) argues that state machines provide a deterministic framework for controlling complex sequences of operations in embedded computing systems.

Advantages:

- **Determinism:** Fixed state transition logic ensure traffic light system will not enter illegal or undefined "blank post".
- **Highly modular:** Each of its posts is independent, making it easy to add new functions. In this traffic light project, we can easy implement traffic flow adaptive logic simply by adding an ADC judgement branch, without overturning the entire original process.
- **Easy to maintain:** Developers can quickly locate and fix logic errors by monitoring status registers (such as curr_state).

Disadvantages:

- **State explosion:** With control complexity increase, state number may rise exponential, increase logic design complexity.
- **Resource occupation:** Complexity state machine code will consume more flip-flops and look-up table resources when running in an FPGA.

1.5 Difference: FPGA vs Microcontroller

Use FPGA and ARM microcontroller to implement FSM in this Assignment, they have essential differences in their operating mechanisms. Chu (2008) emphasizes, compared with microprocessor rely on sequential instruction execution mode, FPGA can implement and run multiple non-interfering state machines simultaneously at the hardware level.

- **FPGA implementation (Physical parallelism):**

  Use Verilog hardware description language, FSM is directly mapped to underlying hardware circuits, including registers and logic gates. In this experiment, ADC data acquisition driver, dynamic nixie tube scanning and core FSM logic run simultaneously and in parallel on the hardware.
- **Microcontroller implementation (Logical parallelism):**

  Use C language, FSM logic usually nest in "switch-case" structure or timer interrupt. CPU must through time-division multiplexing way process tasks in order, cannot avoid interrupt latency and instruction execution turnaround overhead.

Aspects	Microcontroller (e.g., STM32 Cortex-M4)	FPGA (e.g., Altera Cyclone IV)
Speed of Execution	**Sequential Processing:** CPU base on instruction cycle of fetch, decode, execute to run. Although main frequency is very high (can reach hundreds MHz), limited by von Neumann architecture bottleneck, when processing multi-task, it will inevitably produce interrupt latency and context switch overhead (Valvano, 2014).	**Parallel Hardware:** Logic gate circuits work parallel, no instruction read overhead. Although global clock frequency usually lower than high-end MCU, when processing concurrent tasks (like multiple independent FSMs run synchronously), it has real nanosecond-level hardware determinism, throughput is extremely high (Maxfield, 2004).
What can be implemented	Software-Defined Logic: Extremely good at complex conditional branch judgement, general	**Software-Defined Logic:** Extremely good at complex conditional branch

	control, complex mathematical operation and run real-time operating system (RTOS). But its function strictly limited by chip factory 固化的 peripheral modules (like fixed number of ADC, I2C bus etc.) (Valvano, 2014).	judgement, general control, complex math operation and run real-time operating system (RTOS). But its function strictly limited by the pre-programmed peripheral modules when chip leave factory (like fixed number of ADC, I2C bus etc.) (Valvano, 2014).
Commercial Advantages	Single chip cost is extremely low, suit for large-scale high-volume production. It has extremely huge and mature C/C++ software ecology and open-source libraries, greatly shorten product's time-to-market. It also has very good low power consumption sleep mode management ability.	It has extremely strong flexibility and long-life cycle, support product carried out in-the-field hardware reconfiguration after deployment. Compare with develop application specific integrated circuit (ASIC), it no need bear high non-recurring engineering (NRE) cost (Maxfield, 2004).
Commercial Disadvantages	When face massive high-frequency parallel data stream (like high-definition video processing or high-speed multi-axis motor synchronization), it very easy meet performance bottleneck. If underlying hardware exist defect, cannot through software fully repair, must wait next tape-out.	Single chip purchase cost is significantly higher than microcontroller. Development cycle is long, study curve is steep, developers need cost a lot of time solve complex timing closure and wiring congestion problem. Static power consumption usually very big (Monmasson et al., 2011).

Table 1 A Comprehensive Comparison of FPGA and Microcontroller Technologies and Businesses

2. Design Implementation and Development Instructions

2.1 Overall System Overview

This design realizes a pedestrian crossing traffic light control system based on finite state machine. It includes two independent but same function systems:

1) Based on ARM Cortex-M4 STM32L476RG microcontroller, use C language develop;
2) Based on Altera Cyclone IV (EP4CE6E22C8N) development board, use Verilog HDL develop.

The main system functions are as below:

- **Basic traffic light control:** Control motor vehicle red-yellow-green lights and pedestrian red-green lights
- **Pedestrian request response:** Trigger crossing request through buttons on both sides of street
- **Dynamic adjustment according to traffic flow:** Simulate traffic flow through potentiometer voltage collected by ADC, dynamically adjust countdown duration after pressing button, simulate vehicle passing through vehicle sensor, increase countdown time
- **Status indication and interaction:** Display current status, countdown and traffic flow information through LED, 6-digit nixie tube, 0.96-inch OLED screen

2.2 Register-Level Development Implementation Based on STM32

On microcontroller side, in order to pursue higher execution efficiency, this system abandon HAL library, adopt pure register way develop.

- **Clock and Hardware Initialization**

1) **Clock Configuration (RCC):**
Through configure "RCC_PLLCFGR", "RCC_CFGR", "RCC_CFGR" registers, take MSI clock as source, after PLL frequency multiplication, stabilize system clock to 80MHz. Configure FLASH wait cycle, match instruction read timing under high-speed clock, ensure system stable run.

2) **GPIO mapping:**

Pin Number	Function Definition	Direction	Pull-Up/Pull-Down Configuration	Hardware Description / Active Level
PA0	Pedestrian Request Button 1	Input	Pull-Up	Active Low (Triggered when pressed)
PA1	ADC1_IN6 (Traffic Flow Sampling)	Analog Input	None	Connected to potentiometer for analogue signal input
PA4	Buzzer Control	Output	Push-Pull	Initial High Level (Mute), Active Low (Buzzer On)
PA5	Vehicle Green Light	Output	Push-Pull	High Level = Light On
PA6	Vehicle Yellow Light	Output	Push-Pull	High Level = Light On
PA7	Vehicle Red Light	Output	Push-Pull	High Level = Light On
PA8	Pedestrian Request Button 2	Input	Pull-Up	Active Low (Triggered when pressed)
PA9	Vehicle Detection Sensor	Input	Pull-Up	Active Low (Triggered when vehicle is detected)
PB6	Pedestrian Green Light	Output	Push-Pull	High Level = Light On
PB7	Pedestrian Red Light	Output	Push-Pull	High Level = Light On
PB8	I2C1_SCL (OLED Clock Line)	Alternate Function Open-Drain	None	Alternate Function AF4, for I2C communication
PB9	I2C1_SDA (OLED Data Line)	Alternate Function Open-Drain	None	Alternate Function AF4, for I2C communication
PC13	Pedestrian Request Button 3	Input	Pull-Up	Active Low (Triggered when pressed)

Table 2 Pinout Definition

3) **ADC Sampling**

ADC1 (PA1) configure as analogue mode, open hardware self-calibration (ADC_CR_ADCAL). Through 1ms timer interrupt, every 100ms trigger one sampling (adc_req), realize 10Hz sampling frequency. 12-bit sampling value linear map to wait duration ($original_wait_limit = 5000 + (raw_adc_value * 10000) / 4095$), dynamic adjust green light basic wait time.

4) **I2C OLED Driver**

System realizes a set of light-weight I2C driver protocol stack, through direct operate "$I2C1->TXDR$" register and poll "$I2C_ISR_TXIS$" and "$I2C_ISR_STOPF$" flags, realize 0.96-inch OLED initialization and character render, no rely on interrupt. Underlying "$OLED_ShowChar$" function base on $6*8$ $ASCII$ dot-matrix font, enabling real-time updates of the state name, countdown timer, and traffic flow percentage. Finally realize real-time refresh "STATE (current state)", "TIMER (remaining seconds)" and "TRAFF (graphical traffic flow progress bar)". Among them, progress bar through map 0-4095 sampling value equal proportion to 0-50-pixel length, realize intuitive data visualization interaction.

- **FSM Logic**

Microcontroller state machine embeds in "$TIM3_IRQHandler$", take 1ms step carry out state transition:

1) **IDLE:**
Idle state, in this state system default keep vehicle green light always on, at same time detect pedestrian whether send request (PA0, PA8, PC13), according to ADC sampling value calculate basic green light duration (5s - 20s), after calculate finish jump to WAIT state.
WAIT: In this stage keep vehicle green light state. System through poll PA9 pin realize dynamic compensation mechanism (if PA9 be pressed, then automatic add 10s on basic duration), ensure peak hour traffic flow continuity; if no vehicle pass then keep original countdown. After countdown finish, jump to FLASH state.

2) **FLASH:**
Flash green light as switch to yellow light before warning, vehicle area green light takes 250ms as cycle toggle "GPIOA_ODR5" register bit, produce 2Hz green light flash signal, realize green light flash effect, last time is 2s.

3) **YEL1:**
Vehicle yellow light always on 2s, provide necessary brake buffer time for intersection vehicle, after finish enter pedestrian pass phase.

4) **PED:**
In the pedestrian cross mode of pedestrian passing state, it can drive the buzzer to produce 500ms period rhythm sound by the modulo operation. And when countdown only left 3s, the system will automatically shorten the buzzer period to 250ms. The purpose is for through double frequency to remind pedestrians leave the crossing quickly. After the 5s passing is finish, the system force closes the buzzer and clear the timer to zero, then it returns to "IDLE" state, finish the closed loop.

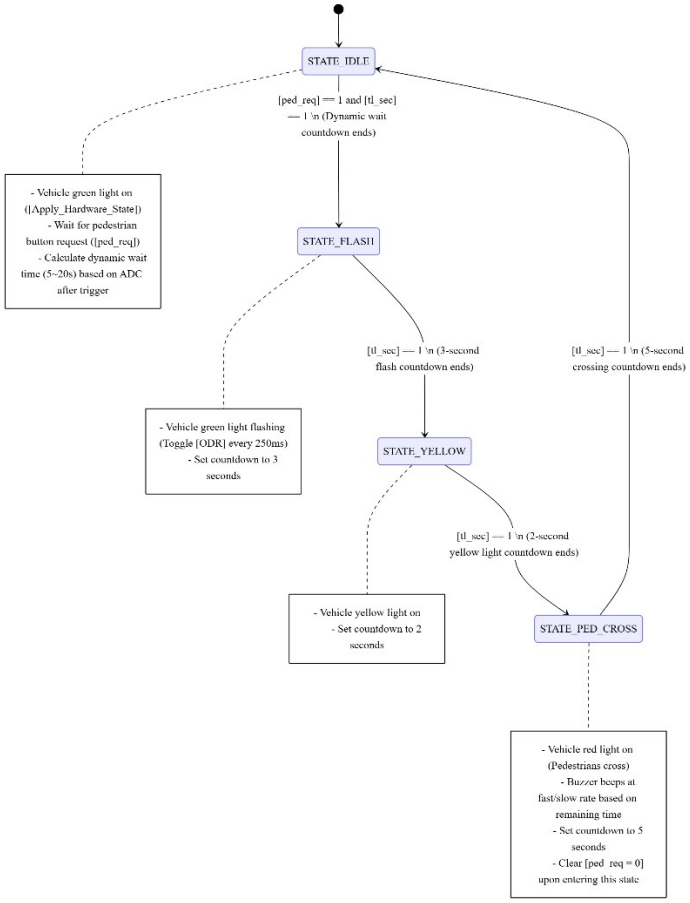


Figure 3. Main traffic light status diagram

2.3 Hardware Logic Implementation Based on FPGA (Verilog HDL)

In FPGA platform design framework, system core no longer same as microcontroller execute by instruction order, but through hardware description language (Verilog HDL) build parallel circuit, realize function.

- **Clock Management and Hardware Mapping Allocation:**

Pin Number	Function Definition	Direction	I/O Standard / Pull-Up	Hardware Description / Active Level
PIN 111	Pedestrian Request 1 (key_ped) / SW9 (sw_flow[0])	Input	3.3V LVTTL / Internal Pull-Up (via constraint)	Dedicated buttons and DIP switches share pins; active-low (triggered by pressing or flipping to a low level)
PIN 142	Pedestrian Request 2 (key_ped2) / SW16 (sw_flow[7])	Input	3.3V LVTTL / Internal Pull-Up (via constraint)	Dedicated buttons and DIP switches share pins; active-low (triggered by pressing or flipping to a low level)

Table 3 FPGA Hardware Pin Assignment

PIN 114, 120, 125, 128, 133, 137	6-bit Flow Switch (sw_flow[6:1])	Input	3.3V LVTTL / Internal Pull-Up (via constraint)	8-bit discrete DIP switch (upper 6 bits) used in place of ADC sampling; active-low (0 when switched on, 1 when released)
PIN 110	Vehicle Red Light (led_r)	Output	3.3V LVTTL / None	Drives the main circuit vehicle red light; Active Low (output logic 0 turns it on)
PIN 113	Vehicle Green Light (led_g)	Output	3.3V LVTTL / None	Triggers the green light for vehicles on the main lane; Active Low (output logic 0 turns it on)
PIN 119	Vehicle Yellow Light (led_y)	Output	3.3V LVTTL / None	Drive the main line vehicle indicator light; Active Low (lit when output logic is 0)
PIN 127	Pedestrian Red Light (ped_led_r)	Output	3.3V LVTTL / None	Triggers the pedestrian crossing signal to turn red; Active High (output logic 1 turns it on)
PIN 132	Pedestrian Green Light (ped_led_g)	Output	3.3V LVTTL / None	Triggers the green light for pedestrians to cross the road; Active Low (output logic 0 turns it on)
PIN 71	Buzzer Control (buzzer)	Output	3.3V LVTTL / None	Drives an active buzzer to provide acoustic feedback; Active High (beeps when the output is logic 1)
PIN 33/34	OLED I2C Bus (oled_scl / oled_sda)	Output / Inout	3.3V LVTTL / Weak Pull-Up (via altera_attribute)	Hardware I2C communication bus with an altera_attribute soft pull-up declaration

Table 3 FPGA Hardware Pin Assignment

- **Core Architecture Selection**

  In the FPGA design, because FPGA and STM32 implementation have different on-chip ADC sampling, so drive external ADC in FPGA need a additional SPI master state machine. The serial wire nature of SPI bus will introduce microsecond-level latency. According to Mano and Ciletti (2018) noted, discrete digital vectors can fundamentally eliminate such risks for state machine control links.
  Therefore, system uses 8-bit DIP switch abstract traffic flows information into 9 (0-8) determined discrete logics. And the module through parallel combinational logic "
  $wire\ [3:0]\ sw_count\ =\ sw_active\ [0]\ +\ \ldots\ +\ sw_active\ [7];$" in single clock cycle (20ns) can complete all pin level summation. When calculate wait time, formula be reconstructed as
  $Wait = 5 + [\frac{sw_count \times 15}{8}]$, in Verilog represent as "$5\ +\ ((sw_count\ *\ 4d'15)\ >>\ 3)$".

- **FSM Core Engine**

Inside FPGA, traffic light finite state machine step has a 26-bit deep counter *"cnt_1hz"* provide precise trigger edge.

This counter accumulates to 49,999,999 under 50MHz clock, produce a 1 second period benchmark, not change any offset with system other part load change.

State machine *"S_GREEN, S_FLASH, S_YELLO, S_RED ∧ so on…"* They are implemented by a D flip-flop array for state latching. In the *S_GREEN* state, the system listens to the *"ped_req"* latch signal through a parallel process. For the 2Hz green light flashing effect in the S_FLASH state, FPGA design saves modulus operation (% 250) of arithmetic unit in code. Hardware directly extracts high bit signal *"cnt_1hz [24]"* of frequency division counter and state bit carry out logic operation: *"~((tl_state == S_GREEN) || (tl_state == S_FLASH && cnt_1hz[24]))"*. This clock bit extraction technique fully demonstrates the elegance of hardware logic when processing periodic signals.

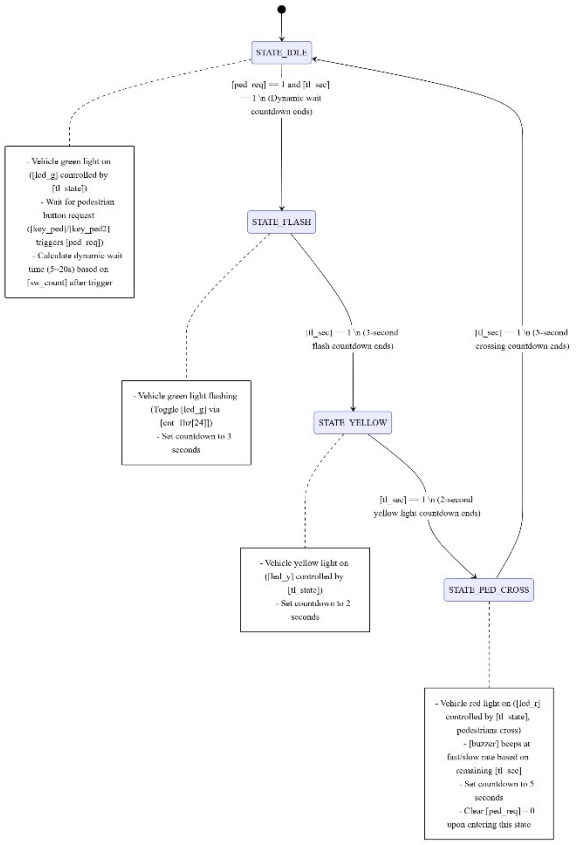


Figure 4 Main traffic light status diagram

- **Fully Independence Parallel I2C and Display Engine**

This traffic light state machine implementation abandons traditional external "*oled_driver.v*" module form, this system cough builds a native I2C controller. Through *"i2c_clk_cnt"* distribute 50MHz clock to suitable working frequency after, system introduce *"i2c_phase"* to precise split every bit setup and timing hold.

For satisfy I2C protocol Wire_AND characteristic, FPGA use three-state buffer control SDA pin: *"assign oled_sda = sda_dir ? sda_out : 1'bz; "*. When need read ACK, sda_dir pull low make pin enter high-impedance state, cooperate internal weak pull-up, allow slave hardware take over bus.

The display driver hardcodes the 40-bit *"get_char"* character font library through a ROM structure. Combine independent m_state display state machine, the video memory data is pushed into the screen in the

form of a data stream. The entire UI rendering link and the core traffic light FSM are physically isolated parallel domains, without blocking each other.

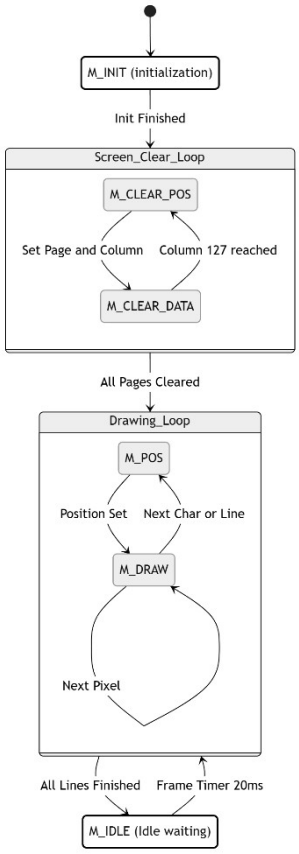


Figure 5 OLED refresh process status diagram

- **Buzzer feedback logic**

Buzzer driver reflects high level trigger hardware characteristic, this module independent with main state machine, use exclusive *"beep_timer"* counter carry out real-time frequency division. When the system enters *"S_RED"* pedestrian pass phase, hardware comparator automatic activate.

When pedestrian crossing countdown island 3s, buzzer signal drive by *"beep_timer[23]"*; when time urgent $\leq 3s$), data selector switch to *"beep_timer[22]"* realize alarm frequency instantaneous double. This adaptive frequency multiplication logic intervention further demonstrates the superiority of FPGA as the core of hard real-time control.

3. Comparison of development processes

3.1 STM32 Microcontroller Development Process

In STM32 development process, development software "Keil" provide powerful debug ability---The debug function. System allows set breakpoint freely in code, and can monitor real-time state of memory variable and register (like real-time check RCC state) this debug process make logic error location very intuitive, as developer can very clearly track FSM every step state transition. This debug system makes system internal state completely transparent, greatly shorten problem location time, is core support of quickly complete function verification in this STM32 development.

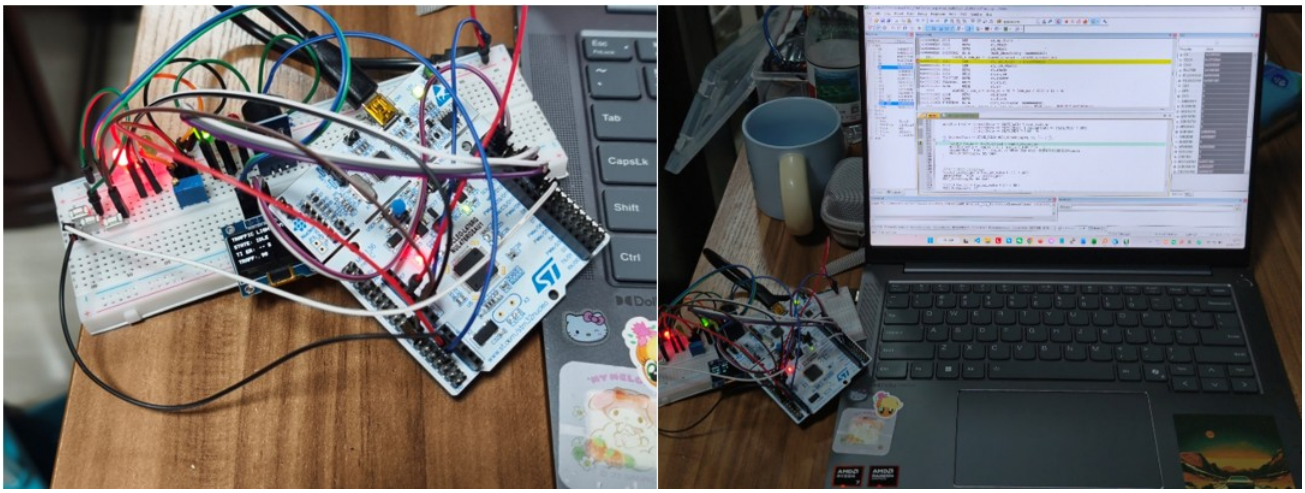


Figure 6 Debugging the STM32 traffic light state machine

3.2 Altera FPGA Development Process

In the FPGA development process is the design process of hardware digital system, the core feature is compiling time longer after code modify, debug mainly is hardware function verification (Maxfield, 2004). In this development, we need in Quartus Prime Lite finish three very cost time steps: Analysis & Synthesis (converting Verilog to a netlist of look-up tables and flip-flops), Fitting (mapping to Cyclone IV EP4CE6 physical resources, typically 3-5 minutes per iteration), and Bitstream Generation (creating a *. sof* configuration file). This process will compare the compile and download in STM32 development cost more time.

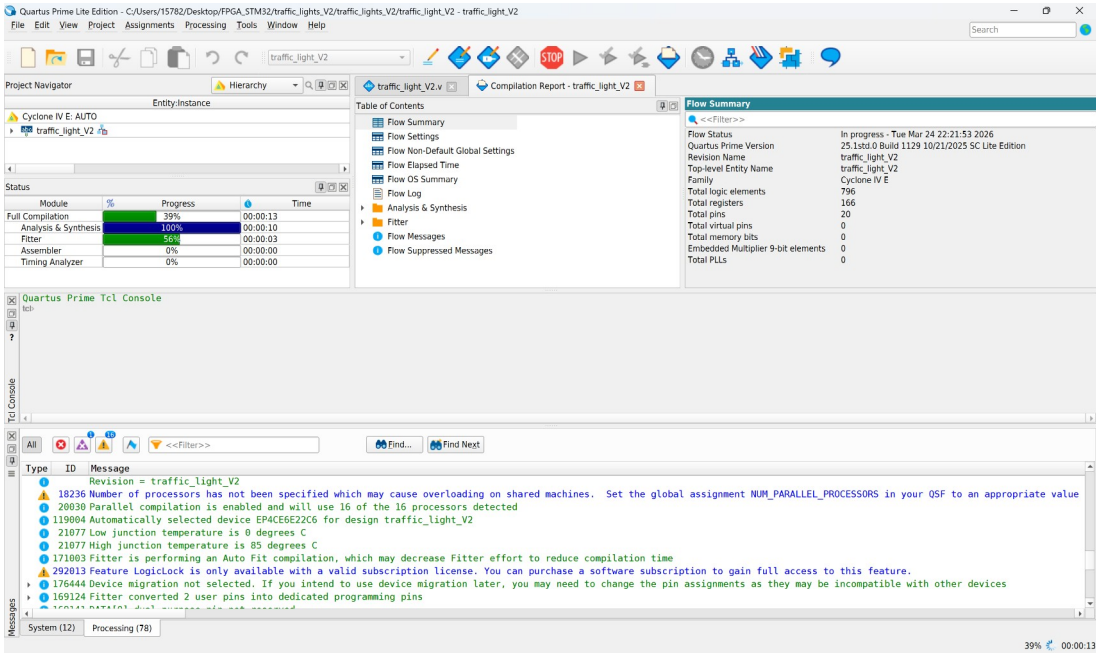


Figure 7 Analysis & Synthesis cost a lot of time

The functional verification used the "compile-download-test" cycle, download to FPGA through JTAG, apply input (pedestrian button, DIP switch), observe output (OLED screen display content, whether the buzzer ring) to confirm FSM state machine the state transition of the FSM, the operation of the I2C controller and acoustic feedback function. If find error, we modify the code and repeat all the process. Since the programming process is very time-consuming, we used a logic analyzer during the testing phase.

- **Hardware Connections and Mapping:** We use the FPGA's reserved I/O pins (Pins 1, 2, 3, 7, 10, 11, 28, and 32) to connect to the logic analyser's channels (D0–D7).
- **Real-time timing verification:** By capturing waveforms using a logic analyser, we directly verified the logic of the FSM during state transitions between S_GREEN, S_FLASH, S_YELLOW, and S_OED_CROSS.
- **Buzzer switching test:** The logic analyser accurately captured the hardware behaviour whereby the buzzer signal frequency doubled from 2 Hz to 4 Hz during the final 3 seconds of the pedestrian crossing phase (S_RED); this would be difficult to quantify by simply observing the phenomenon.

Advantages: This debugging method eliminates the need to repeatedly burn programs to the EEPROM, significantly speeding up the 'modify-compile-verify' iteration cycle. When debugging the core FSM logic, status signals are output via dedicated pins, allowing debugging and verification to be carried out without modifying the I2C driver, OLED, or related code, thereby effectively avoiding interference. Whilst the OLED screen driver is still under development, the logic analyser provides the system with an intuitive feedback interface. There is no need to guess the programme's running state; waveforms can be used to directly confirm whether pedestrian requests have been correctly captured and whether the state machine is transitioning as expected.

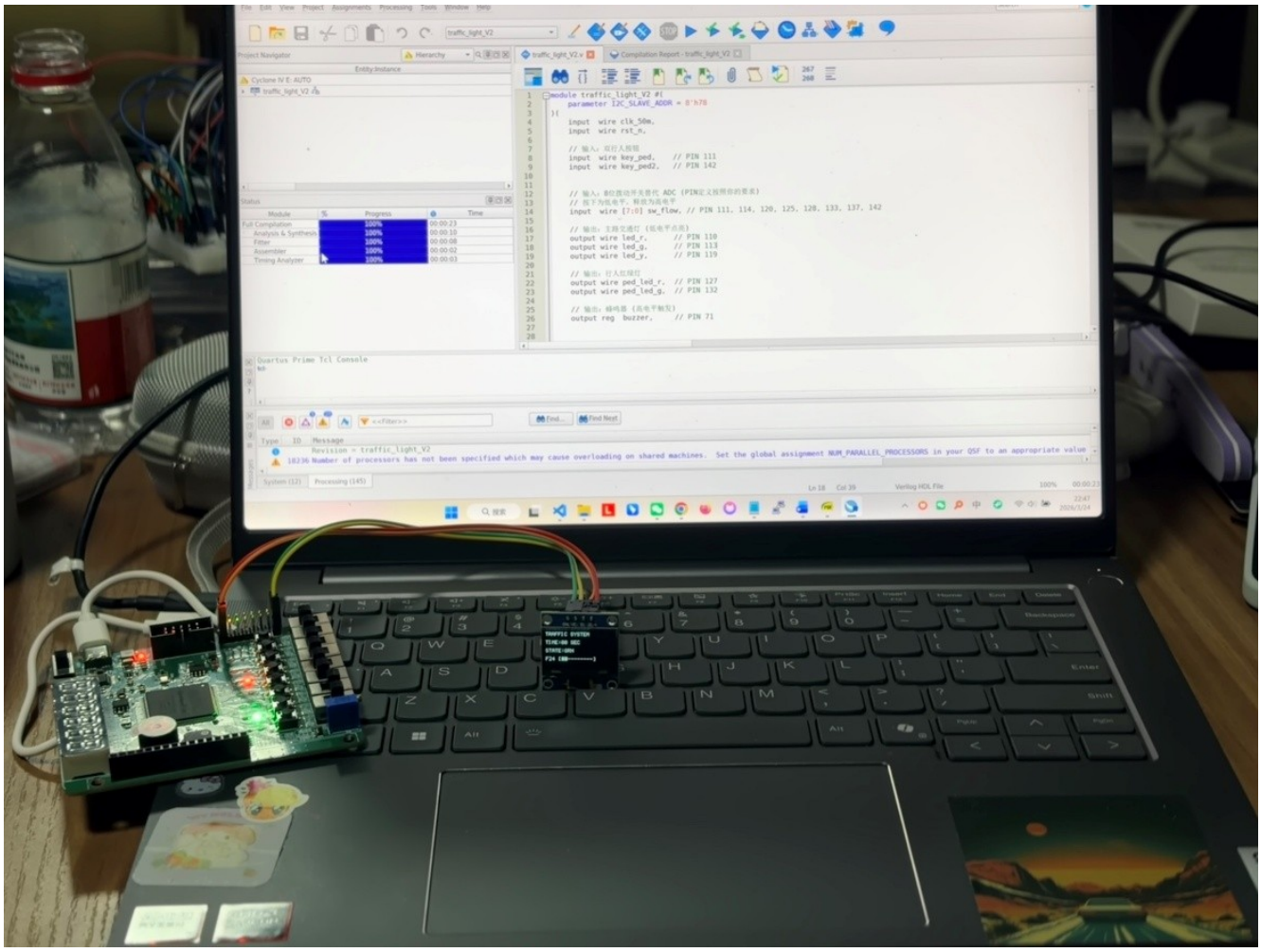


Figure 8 FPGA Traffic Light Implementation

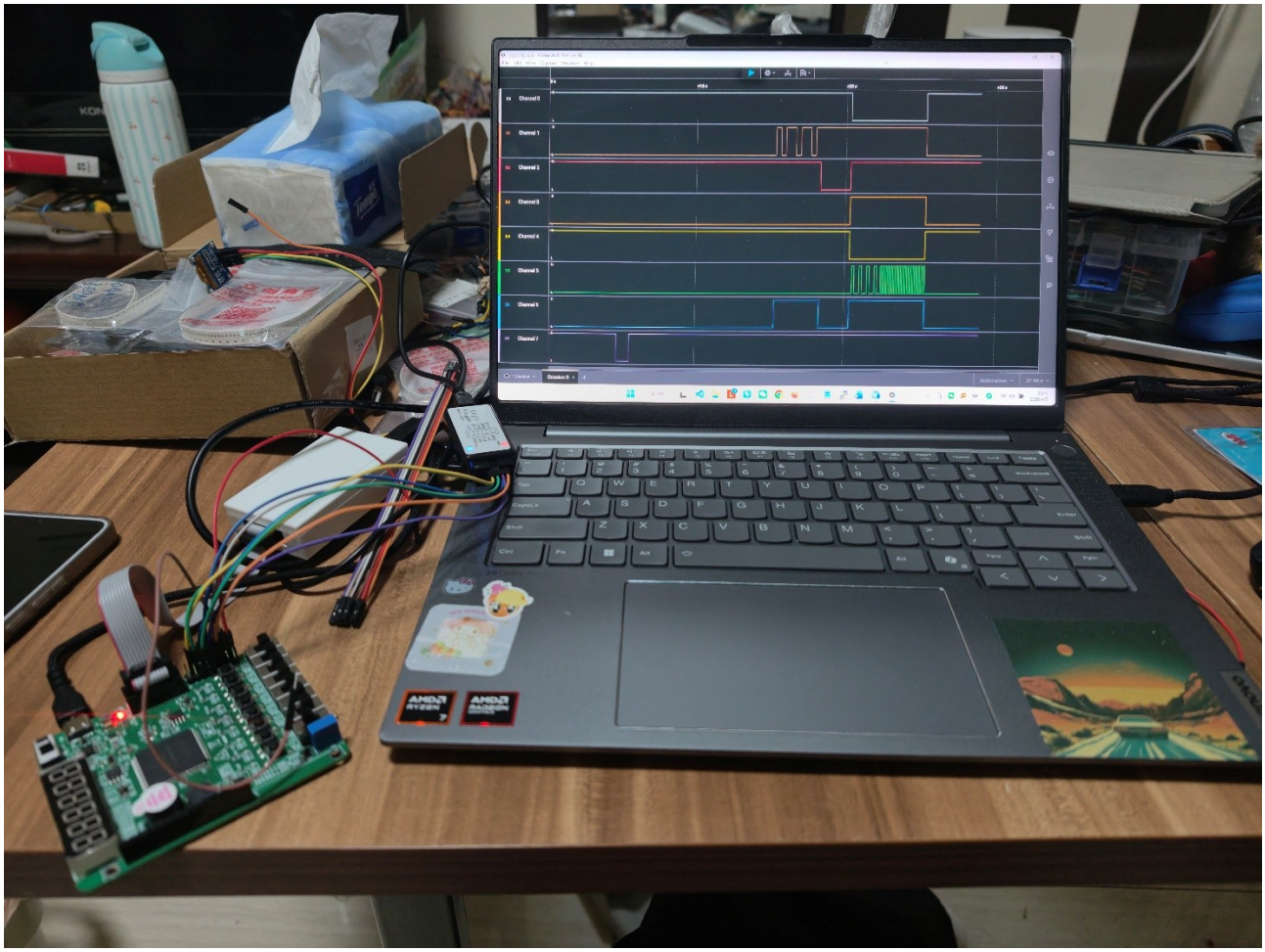


Figure 9 Debugging with a logic analyser

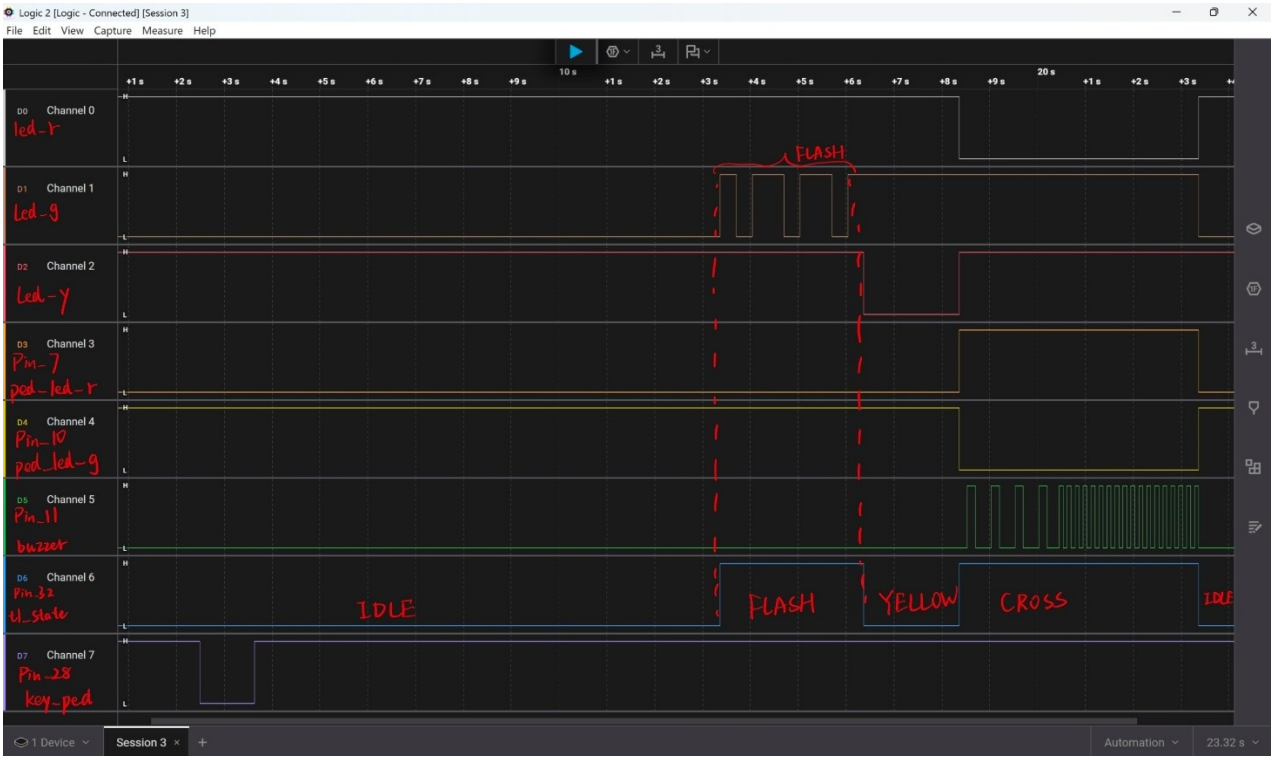


Figure 10 Successful FSM Logic

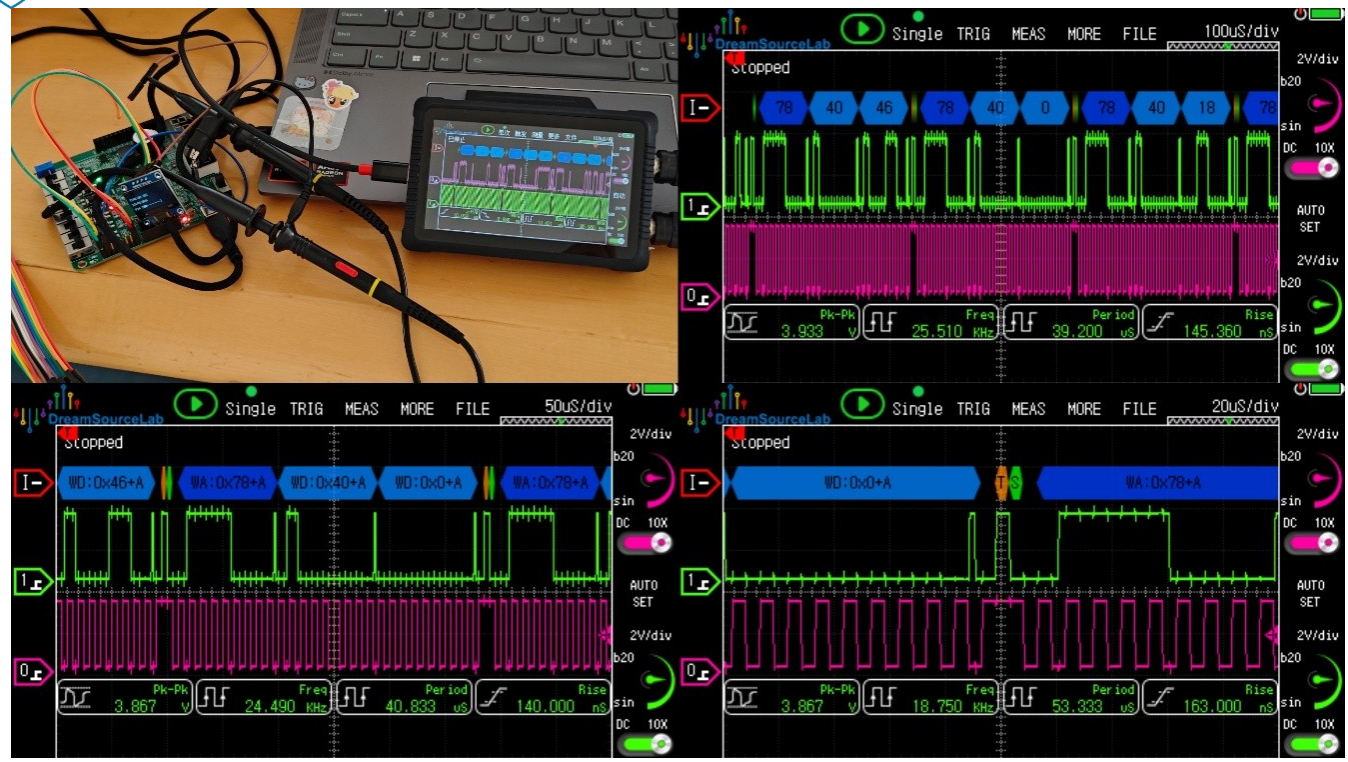


Figure 11 Measuring FPGA I2C Communication with an Oscilloscope

4. Final Verification & Testing

为了验证基于 STM32L476RG 微控制器和 Altera FPGA 双平台的 FSM 以及硬件外设，我们执行了系统性的跨平台测试计划。测试标准包含了系统初始状态，事件驱动跳转，基于传感器（ADC/拨码开关）的动态时长自适应计算，以及硬件反馈是否符合预期

Test number	Function module / Status	Test Actions and Trigger Conditions	Testing Criteria	Actual test results (STM32 & FPGA dual-platform)	Status
T01	System initialization	When the system is powered up or a hardware reset is triggered (pressing the Reset button on the STM32; pulling rst_n low on the FPGA)	The system defaults to STATE_IDLE. The green light for vehicles and the red light for pedestrians remains illuminated. The OLED screen displays 'STATE: IDLE', and the countdown shows '-- S'	The LED initial level is correct. The OLED successfully displays the initial UI and the default traffic percentage	**Pass**
T02	Response to a pedestrian's request to cross the road	In IDLE mode, press the pedestrian request button on either side	When the button is pressed (after debouncing), the internal `ped_req` flag is set to 1. The countdown on the OLED screen begins, with the numbers decreasing, and '-- S' is no longer displayed	Both platforms successfully detected interrupt/level change events; the state machine was successfully activated and began the countdown	**Pass**
T03	Adaptive waiting time based on traffic flow	**STM32:** Adjusting a potentiometer (0–3.3 V) **FPGA:** Set the 8-position flow DIP switch (s_flow). Then press the pedestrian button	The base waiting time `tl_sec` is calculated dynamically: 5 seconds for minimum traffic and 20 seconds for maximum traffic. The TRAFF progress bar on the OLED displays a precise percentage representation of the current physical input	The countdown starts dynamically, ranging from approximately 5 seconds to 20 seconds, based on the ADC sample value and the number of activated DIP switches (sw_count)	**Pass**

T04	IDLE to FLASH state transition	The dynamic wait timer (tl_sec) in the IDLE state is reset to zero	The state machine transitions seamlessly to STATE_FLASH. The internal timer is reset to 3 seconds. The traffic light for motor vehicles flashes at a frequency of 2 Hz (period 500 ms)	状态跳转精准。绿灯物理闪烁 3 秒，OLED 同步显示 "FLASH"。	**Pass**
T05	Flash to YELLOW status transition	The 3-second countdown for the FLASH status has reached zero	The state machine transitions to STATE_YELLOW. The timer is reset to 2 seconds. The amber light for motor vehicles comes on, and the green light goes out	绿灯闪烁结束后立即熄灭，黄灯严格维持 2 秒常亮。	**Pass**
T06	The YELLOW status will transition to CROSS.	The 2-second countdown for the YELLOW phase has reached zero	The state machine transitions to STATE_PED_CROSS. The timer is reset to 5 seconds. The red light for motor vehicles and the green light for pedestrians remain illuminated, and the motor vehicle lane is physically blocked	成功进入行人安全过街相位。红绿信号灯逻辑电平完全符合预期。	**Pass**
T07	Dynamic frequency modulation feedback of the buzzer	During the STATE_PED_CROSS pedestrian crossing phase, continuously monitor the output frequency of the buzzer	**First 2 seconds:** The buzzer emits a slow beep (2 Hz) **Last 3 seconds:** When the timer (tl_sec) is 3 or less, the beeping frequency instantly doubles to a rapid rhythm (4 Hz) to urge pedestrians on	听觉反馈与硬件定时器的高低位提取逻辑（FPGA 的 beep_timer[22]/[23] 及 STM32 模运算）完美契合。	**Pass**
T08	System Status Closed-Loop and Reset	The 5-second countdown for the PED_CROSS pedestrian crossing phase has reached zero	The state machine automatically returns to STATE_IDLE. The pedestrian request flag (ped_req) is cleared. The buzzer is forced to mute. All traffic lights revert to the T01 default state.	系统成功完成了一个完整的闭环 FSM 周期，并稳定等待下一次物理触发。	**Pass**

Table 4. Final Functional Test Form

5. Presentations of the Implemented Systems---Google Drive Link

6.

https://drive.google.com/file/d/16q5nwu4ChkpmFeYQH9bWGupXKoS_Yq4j/view?usp=drive_link

University of Lancashire

Reference List

1. Mano, M. M. and Ciletti, M. D. (2018). *Digital Design: With an Introduction to the Verilog HDL, VHDL, and SystemVerilog*. 6th edn. Upper Saddle River, NJ: Pearson Education.
2. Pedroni, V. A. (2013). *Finite State Machines in Hardware: Theory and Design with VHDL and SystemVerilog*. Cambridge, MA: MIT Press.
3. Roth, C. H. and Kinney, L. L. (2014). *Fundamentals of Logic Design*. 7th edn. Stamford, CT: Cengage Learning.
4. Wolf, M. (2012). *Computers as Components: Principles of Embedded Computing System Design*. 3rd edn. Waltham, MA: Morgan Kaufmann.
5. Chu, P. P. (2008). *FPGA Prototyping by Verilog Examples: Xilinx Spartan-3 Version*. Hoboken, NJ: John Wiley & Sons.
6. STMicroelectronics (2021). *RM0351 Reference manual: STM32L476xx, STM32L486xx, STM32L496xx and STM32L4A6xx advanced Arm®-based 32-bit MCUs*. [online] Available at: https://drive.google.com/file/d/103VTMe8a_8gsGyBkgcjq_J-Htsvr9fVh/view [Accessed: 24 Mar. 2026].
7. Maxfield, C. (2004). *The Design Warrior's Guide to FPGAs: Devices, Tools and Design*. Amsterdam: Elsevier. [online] Available at: https://www.sciencedirect.com/book/monograph/9780750676045/the-design-warriors-guide-to-fpgas [Accessed: 24 Mar. 2026].
8. Monmasson, E., Idkhajine, L., Cirstea, M.N., Bahri, I., Tisan, A. and Naouar, M.W. (2011) 'FPGAs in Industrial Control Applications', *IEEE Transactions on Industrial Informatics*, 7(2), pp. 224-243. Available at: https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=5739129 (Accessed: 24 March 2026)

Appendix:



traffic_lights_V2.zip

Source Project Code 1. FPGA Traffic Light



ARM_Project_Reg_Traffic.zip

Source Project Code 2. STM32 Traffic Light



traffic_lights_simlulation.zip

Source Project Code 3. FPGA debug Project